Forza Fantasy League — Technical Overview

A multi-sport fantasy gaming platform. Web + native mobile, real-time scoring, social leagues, and a peer-to-peer engagement layer.

Document purpose: a concise, high-level briefing for a technical stakeholder. For full detail see the companion "Technical Documentation".

---

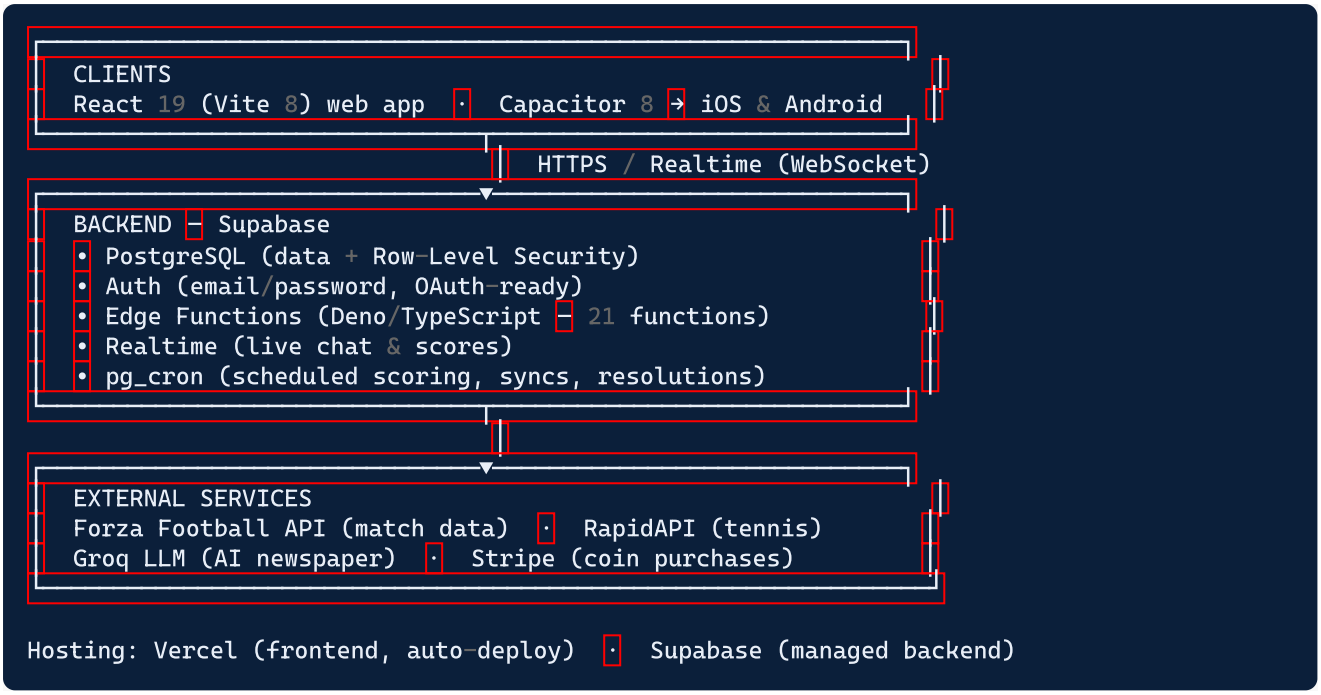
1. What the platform is

Forza Fantasy League is a fantasy-sports platform that lets friends create private leagues, draft or buy squads, compete on live match scoring, and engage through social features (chat, an AI-generated league "newspaper", trading, and betting-style side games). It began as a football product and has been extended into a **multi-sport platform** covering **Football, Formula 1, and Tennis**, with a shared social and competition layer.

Product type	Fantasy sports — web app + iOS/Android (Capacitor wrapper)
Sports supported	Football (live pilot), Formula 1, Tennis
Core loops	Squad building · live scoring · leagues & standings · drafts · trading · side bets · social feed
Monetisation layer	Virtual coin wallet + Stripe purchase path (P2P engagement)
Live status	Football product live with a ~50-user pilot; F1/Tennis/coins built on the platform (`v2`) branch

---

2. Technology stack at a glance



Layer	Technology
Frontend	React 19, Vite 8 (Rolldown bundler), Tailwind CSS 4, React Router 7
Mobile	Capacitor 8 (single codebase → native iOS & Android shells)
Backend	Supabase: PostgreSQL, Auth, Deno Edge Functions, Realtime, pg_cron
Data feeds	Forza Football API (football), RapidAPI ATP/WTA (tennis)
AI	Groq (`llama-3.1`) for the generated league newspaper
Payments	Stripe (virtual coin packs)
Hosting / CI	Vercel (frontend) · GitHub Actions (lint, build, E2E)

3. Architecture in one paragraph

The client is a single-page React application (also wrapped natively for mobile via Capacitor) that talks directly to **Supabase**. Supabase provides the database, authentication, and real-time channels. **Security-sensitive and money/points logic runs server-side** in PostgreSQL functions (called as RPCs) and in Deno Edge Functions, with **Row-Level Security** isolating each user's data. **Scheduled jobs** (pg_cron) drive the live pipeline: they sync fixtures and player data from external sports APIs, ingest live match events, and run the scoring engine that converts real-world performance into fantasy points. Results flow back to users in real time. The frontend auto-deploys to **Vercel** on merge; the backend (functions, migrations) is managed through Supabase.

4. The core systems

System	What it does
Scoring engine	Converts live match stats into fantasy points per position, applies captain multipliers, auto-substitutions, and per-round rules. Idempotent and re-runnable.
Squad & transfer engine	Server-side atomic transfers with budget, club-cap, formation, and transfer-window enforcement.
Draft system	Lottery and reverse-standings drafts, wishlists, knockout-phase keep selections.
Leagues & competition	Classic, draft, head-to-head, and cup formats; standings, ranks, and gazette activity feed.
Trading & auctions	Player-for-player trades and auction listings with bidding and confirmation windows.
Side bets	Commissioner-created prediction bets, auto-resolved against match results.
Coin wallet (P2P)	Virtual-currency wallet with escrow, Stripe top-ups, and an audited transaction ledger.
AI newspaper	A daily LLM-generated league "front page" (headlines, hot takes, rumours) with reactions and comments.
Multi-sport modules	Formula 1 (paddocks, race scoring) and Tennis (player boxes, tournament rosters, ATP Finals pick'em).

5. Data & live pipeline

1. **Sync** — scheduled jobs pull fixtures, players, and statuses from the sports APIs into PostgreSQL.
2. **Ingest** — during matches, a job ingests live events every few minutes and flips fixtures to "live".
3. **Score** — the scoring engine runs every 2 minutes for live fixtures, plus post-match and late-finisher passes, accumulating points per squad per round.
4. **Settle** — once every fixture in a round finishes, the round is marked complete: standings update, head-to-head results resolve, the activity feed is written, and a recovery snapshot is stored.

The pipeline is **idempotent** (safe to re-run) and uses **overlapping schedules** so a missed run is naturally caught by the next.

6. Engineering practices

- **Source control:** GitHub, feature-branch + pull-request workflow; protected `main`.
- **Two-branch model:** `main` (live football pilot) and `v2` (full multi-sport platform — the asset).
- **CI:** GitHub Actions runs lint, production build, and an end-to-end UI smoke suite (Playwright) on every PR.
- **Database:** versioned SQL migrations; append-only discipline; server-side business logic in `SECURITY DEFINER` functions.

- **Security model:** Row-Level Security on user data; protected-column triggers prevent clients from writing money/identity fields directly; sensitive operations run as service-role server-side.
- **Documentation:** an extensive in-repo knowledge base (architecture, API, deployment runbooks, scoring rules) accompanies the code.

---

7. Scale & footprint

Metric	Value
Frontend code	~40,000 lines (React/JSX)
Edge Functions	21 (Deno/TypeScript)
Database migrations	229 SQL files
Screens / routes	~30 (football, F1, tennis, social, admin)
Current users	~50 (football pilot)

---

8. Maturity & roadmap (honest summary)

The platform is **feature-complete across three sports and functional in production** for the football pilot. It is positioned for growth, with a clear, costed hardening roadmap covering: a reproducible/staged environment, expanded automated test coverage of the money/scoring logic, security hardening of a few server endpoints, and observability/alerting. These are documented in detail in the companion remediation backlog and are typical of a product transitioning from a successful pilot to a scale-ready commercial platform.

Companion documents: "Technical Documentation" (full detail) · "Remediation Backlog" (engineering roadmap). Last updated: 2026-06-26.
